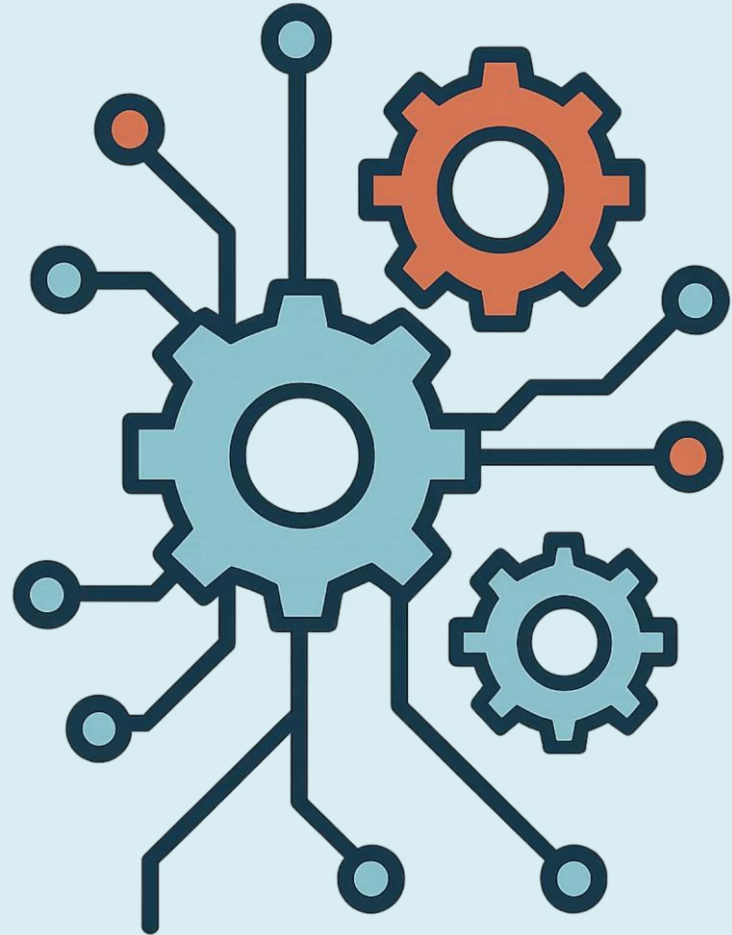




LangChain et l'orchestration des systèmes LLM

Comment structurer, enrichir et piloter des modèles de langage en production — de la gestion des prompts aux agents autonomes.

MODULE 5 – LangChain et orchestration

by ROMAIN ALFRED

Sommaire

01

Pourquoi orchestrer un LLM ?

Les limites fondamentales d'un modèle seul

02

PromptTemplate & Chains

Standardiser et enchaîner les interactions

03

Memory & Retriever

Persistance contextuelle et accès aux données

04

Tools & Agents

Le LLM comme orchestrateur d'actions

05

Synthèse architecturale

Vision globale du pipeline LangChain

Pourquoi orchestrer un LLM ?

Un LLM brut, tel que GPT-4 ou Claude, est un moteur de génération de texte remarquable — mais il souffre de **limitations structurelles fondamentales** qui le rendent insuffisant pour des cas d'usage métier réels.

Sans mémoire

Chaque requête repart de zéro. Le modèle oublie intégralement les échanges précédents.

Sans données externes

Il n'accède pas à vos bases de données, documents internes ou APIs temps réel.

Sans action

Il génère du texte, mais ne peut pas exécuter de code, interroger une API ou déclencher un workflow.

Prompt-dépendant

La qualité de la réponse dépend entièrement de la formulation du prompt — sans structure, les résultats sont imprévisibles.

 LangChain répond à ces quatre limitations en fournissant une couche d'orchestration modulaire, composable et extensible autour des LLMs.

PromptTemplate — Structurer les entrées du modèle

Pourquoi un PromptTemplate ?

Un prompt écrit en dur dans le code est fragile, non réutilisable et difficile à maintenir.
Le **PromptTemplate** de LangChain permet de :

- Séparer la logique métier des données variables
- Réutiliser le même squelette avec des paramètres différents
- Structurer les rôles (system/user) de façon explicite
- Faciliter les tests et le versioning des prompts

Exemple de code

```
from langchain.prompts import PromptTemplate

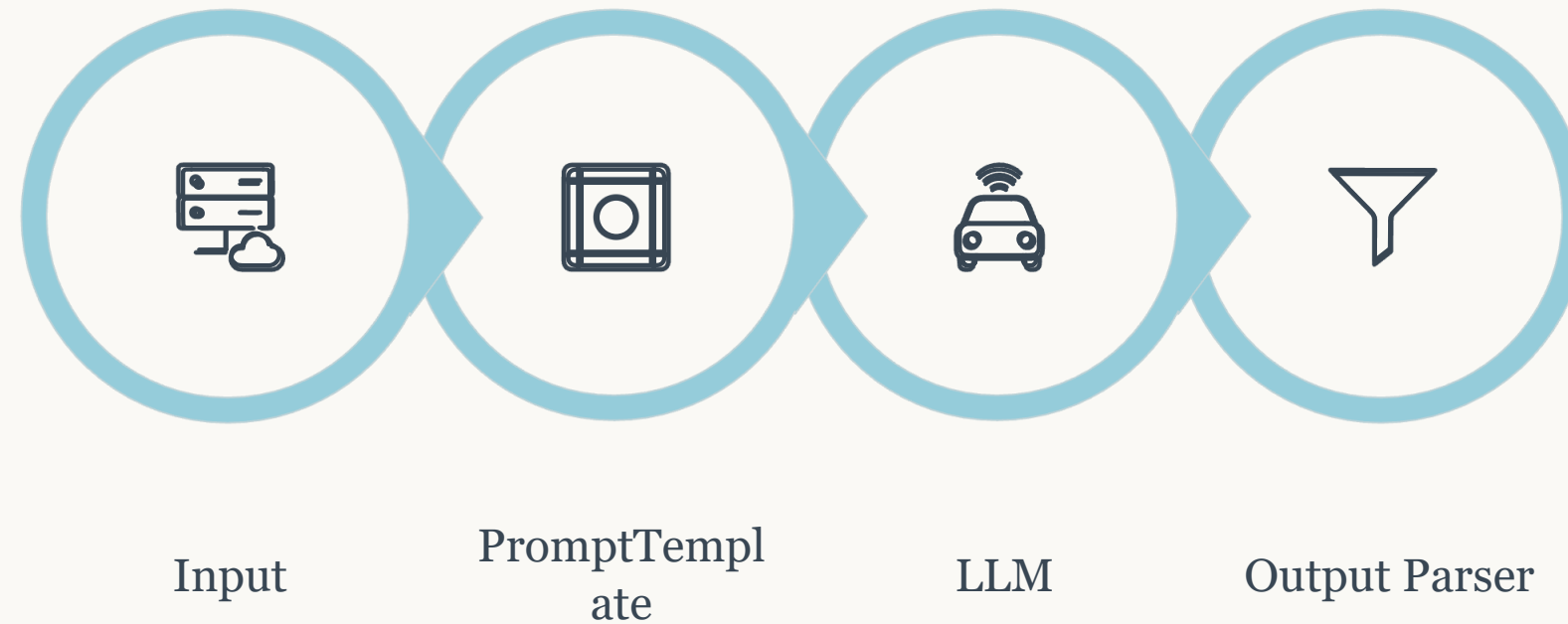
template = PromptTemplate(
    input_variables=["question"],
    template=(
        "Réponds de manière claire "
        "à la question suivante : "
        "{question}"
    )
)

print(template.format(
    question="Qu'est-ce qu'un RAG ?"
))
# → "Réponds de manière claire à la
#     question suivante : Qu'est-ce
#     qu'un RAG ?"
```

- ❏ La variable `{question}` est injectée dynamiquement à l'exécution — le template reste stable.

Chains — Enchaîner les étapes de traitement

Une **chain** est un pipeline séquentiel qui connecte plusieurs composants : un prompt, un LLM, et d'éventuelles étapes de post-traitement. C'est le bloc de base de tout système LangChain non trivial.



Exemple de code — LLMChain

```
from langchain.chains import LLMChain
from langchain.prompts import PromptTemplate

chain = LLMChain(
    llm=llm,
    prompt=template
)

result = chain.run(
    "Explique les transformers"
)
print(result)
```

Types de chains courants

- **LLMChain** : prompt + LLM, le plus simple
- **SequentialChain** : chaîne de plusieurs LLMChains en série
- **RetrievalQA** : chain spécialisée pour le RAG
- **ConversationalChain** : intègre la mémoire automatiquement

Memory — Donner une mémoire au LLM

Le problème : stateless par nature

Un LLM est **sans état** : chaque appel API est indépendant. Sans mécanisme externe, il est impossible de maintenir un fil de conversation cohérent au-delà d'un seul échange.

Les trois types de mémoire

- **Buffer Memory** : conserve l'intégralité de l'historique
- **Window Memory** : ne garde que les N derniers échanges
- **Summary Memory** : résume dynamiquement la conversation pour économiser des tokens

Exemple de code

```
from langchain.memory import (
    ConversationBufferMemory
)

memory = ConversationBufferMemory()

memory.save_context(
    {"input": "Bonjour"},
    {"output": "Salut !"}
)

memory.save_context(
    {"input": "Qu'est-ce que LangChain ?"},
    {"output": "Un framework d'orchestration LLM."}
)

# L'historique est injecté automatiquement
# dans le prochain prompt
print(memory.load_memory_variables({}))
```

- ✓ La mémoire peut être persistée en base de données (Redis, PostgreSQL) pour des sessions longues durée.

Retriever — La couche sémantique vers vos données

Le **Retriever** est l'interface qui permet au LLM d'accéder à des documents externes de façon sémantique. C'est le cœur du pattern **RAG (Retrieval-Augmented Generation)**.

Fonctionnement

→ Question utilisateur

La requête en langage naturel est soumise au retriever.

→ Embedding

La question est convertie en vecteur numérique par un modèle d'embedding.

→ Recherche dans la Vector DB

Les documents les plus proches sémantiquement sont récupérés (top-k).

→ Contexte injecté dans le prompt

Les documents sont fournis au LLM pour générer une réponse ancrée dans vos données.

Distinction clé

⚠ **Retriever ≠ Base de données classique** Un retriever ne fait pas de recherche par mots-clés exacts (SQL, LIKE). Il opère une **similarité vectorielle** : deux phrases de sens proche sont proches dans l'espace vectoriel, même sans mot en commun.

Vector DBs compatibles

- **FAISS** (Meta) — local, rapide
- **Chroma** — léger, open source
- **Pinecone** — cloud, production-ready
- **Weaviate, Qdrant** — alternatives entreprise

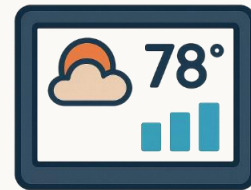
Tools — Connecter le LLM au monde réel

Un **tool** est une fonction externe que le LLM peut appeler pour obtenir des informations ou déclencher des actions. Le modèle cesse d'être un générateur passif pour devenir un **orchestrateur actif**.



Calculatrice

Résoudre des opérations mathématiques complexes que le LLM ne peut pas calculer de façon fiable de tête.



API Météo

Interroger des données temps réel indisponibles dans les données d'entraînement du modèle.



Recherche Web

Récupérer des informations récentes via des moteurs comme Google, Bing ou Tavily Search.



Base SQL

Exécuter des requêtes SQL sur des bases relationnelles et retourner les résultats au modèle.

i Dans LangChain, un tool se définit simplement : une fonction Python décorée avec `@tool`, accompagnée d'une description en langage naturel que le LLM utilise pour décider quand l'appeler.

Agents — Le LLM qui décide et agit

Définition

Un **agent** = LLM + boucle de raisonnement + accès aux tools. Contrairement à une chain (pipeline fixe), un agent **décide lui-même** de la séquence d'actions à exécuter en fonction de la question.

Pattern ReAct

Le pattern dominant est **ReAct** (Reason + Act) : le modèle alterne entre :

- **Thought** : je réfléchis à ce que je dois faire
- **Action** : j'appelle tel tool avec tel paramètre
- **Observation** : je lis le résultat retourné
- **Final Answer** : je synthétise la réponse

Exemple concret

Question : *"Quel temps fait-il à Paris et calcule 23×47 ?"*

1 Appel du tool Météo

L'agent identifie le besoin d'une donnée temps réel et appelle l'API météo avec le paramètre "Paris".

2 Appel du tool Calculatrice

Il détecte un calcul mathématique et délègue à la calculatrice pour un résultat exact (1 081).

3 Synthèse finale

Il combine les deux observations pour formuler une réponse cohérente et complète.

Synthèse — L'architecture complète LangChain

Tous les composants vus en formation s'articulent en un pipeline cohérent. Chaque couche joue un rôle précis et complémentaire pour transformer une question brute en réponse augmentée et fiable.



Ce que LangChain apporte

- Modularité : chaque composant est interchangeable
- Composabilité : les briques s'assemblent librement
- Observabilité : LangSmith permet le traçage complet

Prochaines étapes recommandées

- Expérimenter avec LangChain Expression Language (LCEL)
- Construire un premier RAG sur vos propres documents
- Explorer LangGraph pour les agents multi-étapes complexes